JavaScript 옵저버 패턴 학습 — 1일차 진행 중

오늘의 결론

옵저버 패턴은 한 객체의 변경을 여러 객체에 느슨하게 전달하는 구조다. 핵심은 Subject가 Observer 목록을 관리하고, Observer가 구독·해제를 명시적으로 관리하는 것이다.

핵심 구조

- Subject: 상태나 이벤트를 소유하고 알림을 발행한다.
- Observer: 알림을 구독하고 이벤트를 처리한다.
- subscribe: 구독을 등록한다.
- unsubscribe: 더 이상 알림을 받지 않도록 제거한다.
- notify: 모든 Observer에게 이벤트를 전달한다.

소스 파일

- `/workspace/observer-pattern/observer.js`
- `/workspace/observer-pattern/observer-demo.mjs`

실행

```bash
node /workspace/observer-pattern/observer-demo.mjs
```

읽어야 할 부분

1. `Set`으로 Observer 중복 등록을 방지하는 이유를 설명한다.
2. `subscribe`가 해제 함수를 반환하는 이유를 설명한다.
3. `notify`에서 `[...this.#observers]`로 복사한 뒤 순회하는 이유를 생각한다.
4. 이벤트 객체를 직접 전달할 때 Observer가 Subject 내부 상태를 훼손하지 않도록 하려면 무엇을 추가해야 하는지 생각한다.

실습 과제

`NewsPublisher`를 구현한다.

- `publish(title, content)` 메서드를 만든다.
- 구독자는 `{ type: "NEWS_PUBLISHED", title, content }` 이벤트를 받는다.
- 같은 함수를 두 번 구독해도 한 번만 호출되어야 한다.

unsubscribe 후에는 이벤트를 받지 않아야 한다.

잘못된 Observer 등록은 TypeError로 거부한다.

완료 조건

옵저버 패턴의 Subject와 Observer 역할을 자신의 말로 설명했다.

제공된 예제를 직접 실행했다.

NewsPublisher 실습을 구현했다.

구독·알림·해제를 검증하는 테스트 결과를 제출했다.

메모리 누수와 구독 해제의 관계를 설명했다.

완료 조건을 확인하기 전에는 2일차로 진행하지 않는다.